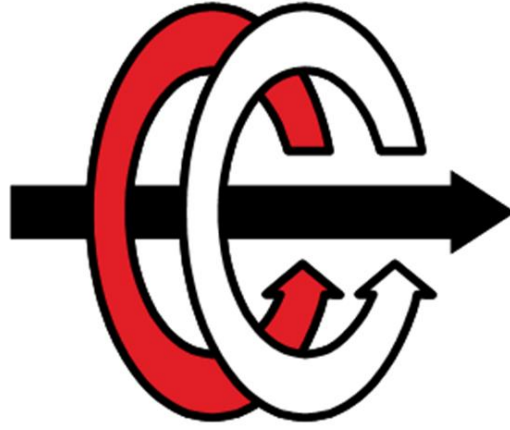




EE 374

**Fundamentals of Power Systems and
Electrical Equipment**

Term Project

Ece Simge PETEK

2443711

Birkan GENÇ

2443059

Date Of Submission: 09.06.2024

1.Introduction

The transmission network is a critical component of power systems, serving as the backbone that connects electricity generation and distribution to deliver electrical energy to end-users. These networks, often extending hundreds of kilometers, rely on overhead transmission lines supported by transmission line towers. The design of these lines and towers is influenced by various factors such as power ratings, voltage levels, and available equipment. Conductors for each phase are bundled together, and the geometric configuration of these bundles is carefully planned to ensure optimal performance. The spacing between phases and the neutral line depends on several factors, including environmental conditions, power and voltage ratings, the number of conductors in a bundle, and the number of circuits on the tower. Transmission towers also serve multiple purposes, such as grounding lightning strikes, carrying telecommunication lines, and ensuring safety with visible markers for aircraft.

The project “Transmission Line Tower Design Calculator” will be an essential tool for engineers worked in the planning and construction of electrical power transmission lines. This project, developed using Python, features a user-friendly graphical user interface (GUI) that simplifies the complex calculations required for designing efficient and safe transmission systems. By including a range of configurations and design specifications, the calculator proves to be a good resource for electrical engineering professionals.

2.Project Description

The Transmission Line Tower Design Calculator provides a comprehensive solution for determining the parameters of transmission lines and towers. The program starts by allowing users to select from three distinct tower types: Narrow Base Tower (Type-1), Single Circuit Delta Tower (Type-2), and Double Circuit Vertical Tower (Type-3). Each tower type has specific geometric constraints and voltage levels that are considered during the calculations. For the Double Circuit Vertical Tower (Type-3), users can specify whether a single or double circuit configuration will be used, adding flexibility to the design process.

After selecting the tower type, users input the X-Y coordinates of the phase lines relative to the tower's ground center. These coordinates are crucial for determining the spatial arrangement of the conductors and must obey to the specified geometric limitations for each tower type. The program checks these inputs to ensure they are within acceptable ranges and issues warnings or error messages if any inputs are invalid. This feature is particularly important for maintaining the integrity of the design and ensuring that the calculations are based on feasible configurations.

Users can also specify the number of conductors in each bundle (up to a maximum defined by the tower type) and the distance between conductors within the bundle. The distance between conductors is assumed to form a regular polygon, which simplifies the calculations for inductance and capacitance. Additionally, the program supports five different conductor types—Hawk, Drake, Cardinal, Rail, and Pheasant—each with distinct technical properties such as diameter, GMR, AC resistance, and current capacity.

Once the transmission line length is specified, the program performs calculations to determine the line resistance (R), line inductance (L), line charging capacitance (C), and line capacity (MVA). These calculations take into account the user inputs and the physical properties of the selected conductors, providing accurate and reliable results. The calculated parameters are then displayed within the GUI, making it easy for users to interpret the results.

To conclude, the "Transmission Line Tower Design Calculator" simplifies the process of designing transmission lines and towers. By integrating detailed input requirements, validation checks, and detailed calculations, the tool supports efficient and accurate power transmission planning.

3. Tower Types

1) Type-1: Narrow Base Tower: It is primarily utilized for lower voltage levels and in areas where land costs are high.

Geometric Configuration:

- Maximum Height for a Phase Line: 39 meters
- Minimum Height for a Phase Line: 23 meters
- Maximum Available Horizontal Distance from the Tower Center: 4 meters
- Minimum Available Horizontal Distance from the Tower Center: 2.2 meters

Voltage Level: 66 kV

Maximum Number of Conductors Inside the Bundle: 3

The Narrow Base Tower (Type-1) is distinguished by its asymmetrical placement of phase lines, a feature designed to optimize space utilization while maintaining efficient power transmission. Designed to withstand the specified voltage level, this tower type ensures cost-effective transmission in areas where land availability is limited. The maximum and minimum heights for phase lines are carefully determined to provide the necessary vertical aperture for conductors, considering various factors such as terrain characteristics and ground use. Horizontal distances from the tower center play a critical role in determining the spacing between phase lines, a factor essential for preserving electrical safety and reducing corona discharge risks. Moreover, the maximum number of conductors inside the bundle sets the transmission line's capacity, influencing key parameters such as line resistance and current carrying capability. In summary, the Narrow Base Tower (Type-1) presents a compact and efficient solution for transmitting electrical power at lower voltage levels, addressing the challenges posed by space constraints and high land costs in its operational environment.

GMD Calculation:

$$\begin{aligned}D_{ab} &= \sqrt{(y_a - y_b)^2 + (x_b - x_a)^2} \\D_{bc} &= \sqrt{(y_b - y_c)^2 + (x_c - x_b)^2} \\D_{ca} &= \sqrt{(y_a - y_c)^2 + (x_a - x_c)^2} \\GMD &= \sqrt[3]{D_{ab} \times D_{bc} \times D_{ca}}\end{aligned}$$

2) Type-2: Single Circuit Delta Tower: It represents a significant advancement in transmission tower design, offering enhanced capabilities for handling higher voltage levels compared to its predecessor, the Narrow Base Tower. This tower type is distinguished by its horizontal placement of phase lines, a feature that optimizes its suitability for applications requiring robust power transmission at elevated voltage levels.

Geometric Configuration:

- Maximum Height for a Phase Line: 43 meters
- Minimum Height for a Phase Line: 38.25 meters
- Maximum Available Horizontal Distance from the Tower Center (for the side phases): 11.5 meters
- Minimum Available Horizontal Distance from the Tower Center (for the side phases): 9.4 meters
- Maximum Available Horizontal Distance from the Tower Center (for the center phase): 8.9 meters

Voltage Level: 400 kV

Maximum Number of Conductors Inside the Bundle: 4

The Single Circuit Delta Tower (Type-2) features a horizontal placement of phase lines, optimizing its capacity to handle higher voltage levels efficiently. With a maximum height of 43 meters and a minimum height of 38.25 meters for phase lines, this tower type ensures enough vertical clearance for conductors, considering various factors such as terrain and ground use. Horizontal distances from the tower center differ for side phases and the center phase, determining the spatial arrangement of conductors and ensuring optimal electrical performance. Its horizontal placement of phase lines offers advantages in terms of ease of installation and maintenance, as well as improved stability under high wind and seismic conditions. The maximum number of conductors permitted inside the bundle is set at four, enhancing the tower's capacity and performance for transmitting higher voltages. Overall, the Single Circuit Delta Tower (Type-2) presents a robust solution for transmitting electrical power at elevated voltage levels, offering enhanced efficiency and reliability in applications where increased voltage capacity is required.

GMD Calculation: It is the same as Type-1: Narrow Base Tower.

3) Type-3: Double Circuit Vertical Tower: It is specifically engineered to accommodate double circuit configurations, offering versatility and scalability to meet evolving power transmission needs. This tower type is designed to provide efficient and reliable transmission capabilities, whether deploying a single or double circuit, depending on future planning and investment considerations.

Geometric Configuration:

- Maximum Height for a Phase Line: 48.8 meters
- Minimum Height for a Phase Line: 36 meters
- Maximum Available Horizontal Distance from the Tower Center: 5.35 meters
- Minimum Available Horizontal Distance from the Tower Center: 1.8 meters

Voltage Level: 154 kV

Maximum Number of Conductors Inside the Bundle: 3

The Double Circuit Vertical Tower (Type-3) is specially designed for double circuit configurations, offering flexibility for future expansion and increased transmission capacity. With heights ranging from 36 meters to 48.8 meters, it ensures enough vertical clearance for conductors across varied terrain

profiles. Horizontal distances from the tower center, between 1.8 meters and 5.35 meters, optimize phase line spacing for structural integrity and electrical performance. Operating at 154 kV, it efficiently delivers power while conserving space and resources despite its vertical design, making it a cost-effective solution for modern transmission networks.

GMR & GMD Calculation

For the single circuit case:

$$\begin{aligned} D_{ab} &= \sqrt{(y_a - y_b)^2 + (x_b - x_a)^2} \\ D_{bc} &= \sqrt{(y_b - y_c)^2 + (x_c - x_b)^2} \\ D_{ca} &= \sqrt{(y_a - y_c)^2 + (x_a - x_c)^2} \\ GMD &= \sqrt[3]{D_{ab} \times D_{bc} \times D_{ca}} \end{aligned}$$

For the double circuit case:

$$\begin{aligned} D_{sa} &= \sqrt{GMR_{bundle} \times \sqrt{(y_a - y_{a2})^2 + (x_a - x_{a2})^2}} \\ D_{sb} &= \sqrt{GMR_{bundle} \times \sqrt{(y_b - y_{b2})^2 + (x_b - x_{b2})^2}} \\ D_{sc} &= \sqrt{GMR_{bundle} \times \sqrt{(y_c - y_{c2})^2 + (x_c - x_{c2})^2}} \\ GMR_{bundle} &= \sqrt[3]{D_{sa} \times D_{sb} \times D_{sc}} \end{aligned}$$

For both cases:

$$\begin{aligned} D_{ab} &= \sqrt{\sqrt{(y_a - y_b)^2 + (x_b - x_a)^2} \times \sqrt{(y_a - y_{b2})^2 + (x_a - x_{b2})^2}} \\ D_{bc} &= \sqrt{\sqrt{(y_b - y_c)^2 + (x_c - x_b)^2} \times \sqrt{(y_b - y_{c2})^2 + (x_b - x_{c2})^2}} \\ D_{ca} &= \sqrt{\sqrt{(y_a - y_c)^2 + (x_c - x_a)^2} \times \sqrt{(y_a - y_{c2})^2 + (x_a - x_{c2})^2}} \\ GMD &= \sqrt[3]{D_{ab} \times D_{bc} \times D_{ca}} \end{aligned}$$

Transmission Line Capacity Calculations

For every Tower Type below equation is used:

$$Capacity (MVA) = \sqrt{3} * Conductor Capacity * Number of Conductors * \frac{Voltage Level}{1000000}$$

Additionally, the capacity is multiplied with 2 if double circuit option is chosen.

Bundle Geometric Mean Radius (GMR) and Equivalent Radius Calculations

For a Bundle with 4 Conductors: Since the shape of the bundle is like a square with 4 conductors, we need to use a different formula different than 1, 2, 3 conductors' cases.

$$GMR_{bundle} = (Conductor\ GMR * (Distance\ Between\ Conductors)^3 * \sqrt{2})^{1/4}$$

Similarly, the equivalent radius of the bundle (r_{eq}) is calculated as:

$$r_{eq_bundle} = (Conductor\ Radius * (Distance\ Between\ Conductors)^3 * \sqrt{2})^{1/4}$$

For a Bundle with 1,2 or 3 Conductors:

$$GMR_{bundle} = (Conductor\ GMR * (Distance\ Between\ Conductors)^{number\ of\ conductors-1} * \sqrt{2})^{\frac{1}{number\ of\ conductors}}$$

Similarly, the equivalent radius of the bundle (r_{eq}) is calculated as:

$$r_{eq_bundle} = (Conductor\ Radius * (Distance\ Between\ Conductors)^{number\ of\ conductors-1} * \sqrt{2})^{\frac{1}{number\ of\ conductors}}$$

Transmission Line Parameters Calculations

1) Resistance (R)

The resistance of the transmission line is calculated using the following formula:

$$R = \frac{Conductor\ Resistance}{Number\ of\ Conductors} \text{ (ohms per km)}$$

Then we multiply the result with transmission line length to find the total resistance.

2) Inductance (L)

$$L = 2 * 10^{-7} * \log \frac{GMD}{GMR_{bundle}} * 10^3 \text{ (mH per meter)}$$

We multiply the result with transmission line length (in meters) to find the total inductance of the line in mH.

3) Capacitance (C)

$$\frac{2\pi * 8.854 * 10^{-12}}{\log \frac{GMD}{GMR_{bundle}}} * 10^6 \text{ (\mu F per meter)}$$

We multiply the result with transmission line length (in meters) to find the total capacitance of the line in μF .

4. The Design Steps of Program

The code of the program is included in the Appendix section. The stages of the program are explained below.

- **Importing Required Modules and Libraries:** Imports necessary modules and libraries for designing the GUI using PyQt's Pyside6 version.
- **Setting Dark Theme:** The `set_dark_theme` function sets the application to use a dark theme.
- **Coordinate Limits:** A dictionary named `limits` contains coordinate limits for different types of towers. This is used to validate the coordinates entered by the user.
- **Definition of GUI Class:** Defines a class named `TransmissionLineGUI` which inherits from the `QMainWindow` class. This class defines the appearance and behaviors of the GUI.
- **Creating the User Interface:** The `initUI` method creates the interface elements and places them. These elements include labels, buttons, line edits, combo boxes, and frames.
- **Updating Inputs for Circuits:** The `update_circuits_input` method updates the input fields and coordinate limits based on the selected tower type. It also enables or disables additional input fields based on whether the user can select a second circuit.
- **Displaying Images:** The `initUI` method shows an image based on the selected tower type. This provides a visual representation of the design corresponding to the user's selection.
- **Connection with Calculation Button:** When the calculation button is clicked, the `calculate_parameters` function is called. This function calculates the transmission line parameters using the entered values and writes the results to the output fields.

These steps demonstrate how to create a GUI application using PyQt, allowing users to easily calculate transmission line design parameters.

- **validate_input function:** This function is responsible for validating the user inputs. It checks for empty fields, ensures numeric values are entered, and verifies the validity of coordinates. If any input is invalid, it highlights the respective fields in red and displays error messages. If all inputs are valid, it returns `True`; otherwise, it returns `False`.
- **get_conductor_parameters function:** This function returns the parameters of the selected conductor type based on the user's choice. Parameters include conductor radius, GMR (Geometric Mean Radius), AC resistance, and current capacity. These values are used in the subsequent calculations.
- **calculate_parameters function:** This function calculates various parameters of the transmission line based on the user inputs and selected options. It first validates the inputs using the `validate_input` function. If the inputs are valid, it proceeds to calculate parameters such as resistance (R), inductance (L), capacitance (C), and line capacity. These parameters are then displayed in the appropriate output fields in the GUI.
- **Main Execution:** The script starts the `QApplication`, sets the dark theme, initializes the `TransmissionLineGUI` window, shows it to the user, and starts the application event loop using `app.exec()`.

These steps demonstrate handling user inputs, calculating parameters, and displaying results in a PyQt GUI application for transmission line analysis.

Executable: To convert our Python script into an executable program, we used the "PyInstaller" library. First, we installed the library. Then, we ran the following command in the terminal:

```
pyinstaller --onefile -w [our_script_name.py]
```

“--onefile” option packages everything into a single executable file, making it easier to distribute and use. And “-w” option tells PyInstaller to create a windowed application, suppressing the console window. This is particularly useful for GUI applications. After running this command, PyInstaller generates an executable file from our script, allowing it to be run independently of Python.

5. Test Results & Discussion

- Figure 1., Figure 2. and Figure 3. show the default interfaces when each tower type is selected.

Transmission Line Design Calculator

Tower Type: **Type-1: Narrow Base Tower**

Number of Circuits: **1**

X-Y Coordinates of the Phase Lines (m):

	1. Circuit X-Y Coordinates	2. Circuit X-Y Coordinates
Phase A, X coordinate:	-4.0 - -2.2	N/A
Phase A, Y coordinate:	23 - 39	N/A
Phase B, X coordinate:	2.2 - 4.0	N/A
Phase B, Y coordinate:	23 - 39	N/A
Phase C, X coordinate:	2.2 - 4.0	N/A
Phase C, Y coordinate:	23 - 39	N/A

Number of Conductors in Bundle: **1 - 3**

Distance Between Conductors in Bundle (cm):

Conductor Type: **Hawk**

Length of Transmission Line (km):

Calculate

Line Resistance R (Ω):

Line Inductance L (mH):

Line Charging Capacitance C (μF):

Line Capacity (MVA):

Type-1: Narrow Base Tower

Figure 1. The Default Interface When Type 1-Narrow Base Tower Is Selected.

Transmission Line Design Calculator

Tower Type: **Type-2: Single Circuit Delta Tower**

Number of Circuits: **1**

X-Y Coordinates of the Phase Lines (m):

	1. Circuit X-Y Coordinates	2. Circuit X-Y Coordinates
Phase A, X coordinate:	-11.5 - -9.4	N/A
Phase A, Y coordinate:	38.25 - 43	N/A
Phase B, X coordinate:	-8.9 - 8.9	N/A
Phase B, Y coordinate:	38.25 - 43	N/A
Phase C, X coordinate:	9.4 - 11.5	N/A
Phase C, Y coordinate:	38.25 - 43	N/A

Number of Conductors in Bundle: **1 - 4**

Distance Between Conductors in Bundle (cm):

Conductor Type: **Hawk**

Length of Transmission Line (km):

Calculate

Line Resistance R (Ω):

Line Inductance L (mH):

Line Charging Capacitance C (μF):

Line Capacity (MVA):

Type-2: Single Circuit Delta Tower

Figure 2. The Default Interface When Type 2- Single Circuit Delta Tower Is Selected.

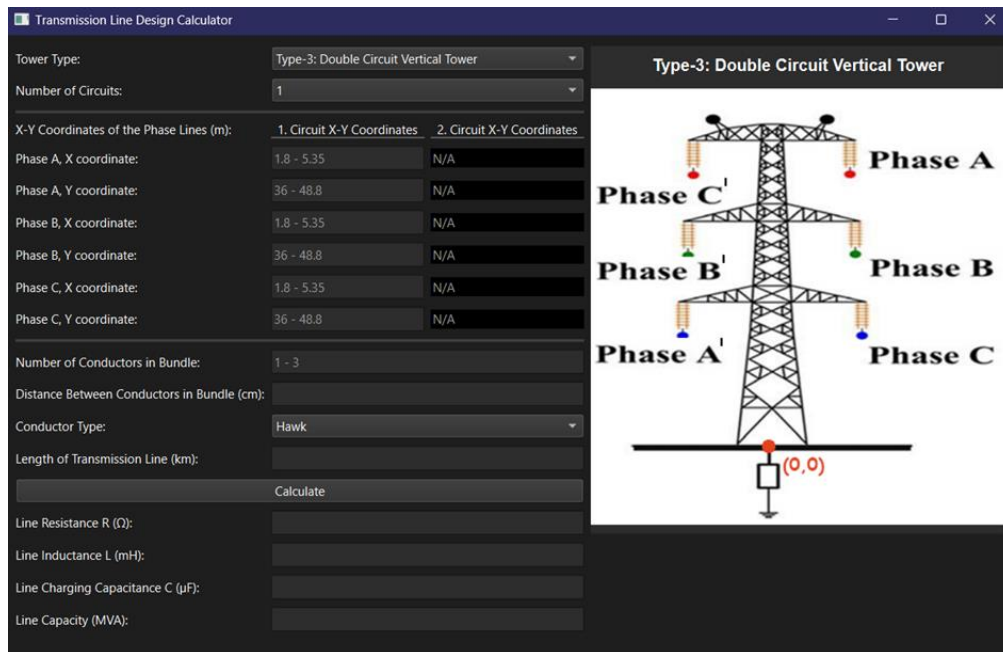


Figure 3. The Default Interface When Type 3- Double Circuit Vertical Tower Is Selected

Figure 4. shows the error given by the program when the "Calculate" button is pressed when no input is entered.

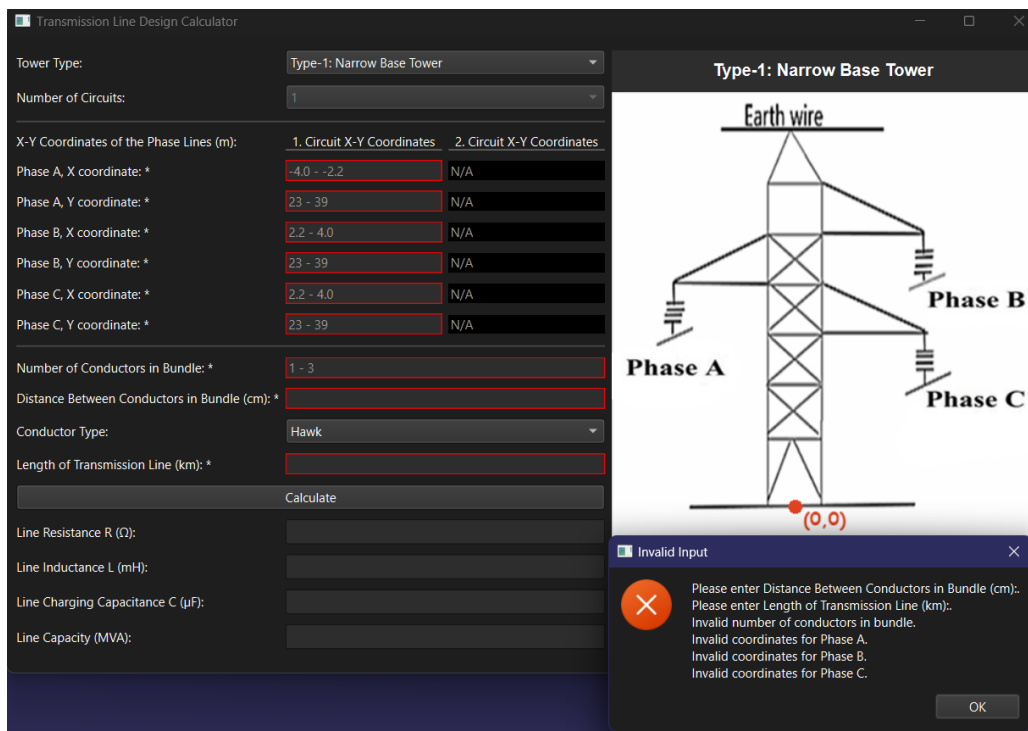


Figure 4. The Interface And Error Message When There Is No Input.

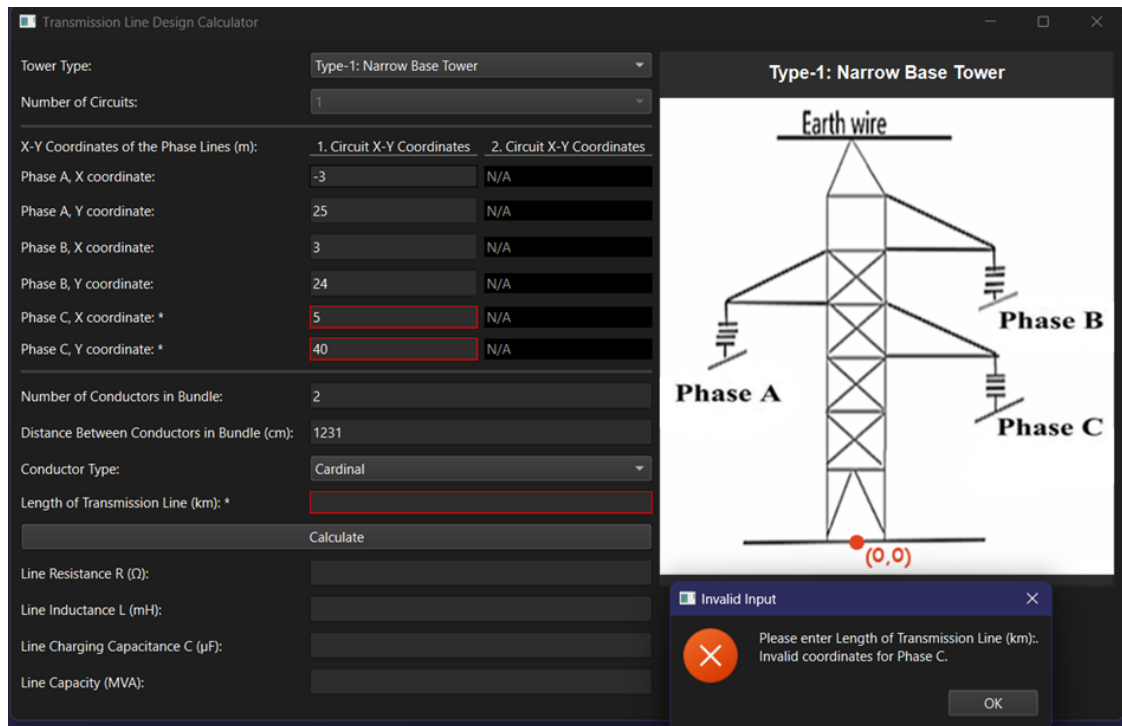


Figure 5. The Interface And Error Message When The Partially Correct Input Is Entered.

When correct inputs are entered, Figure 6 and Figure 7 shows the calculation of R(resistance), L(inductance), C(capacitance) and line capacity according to the selected tower type.

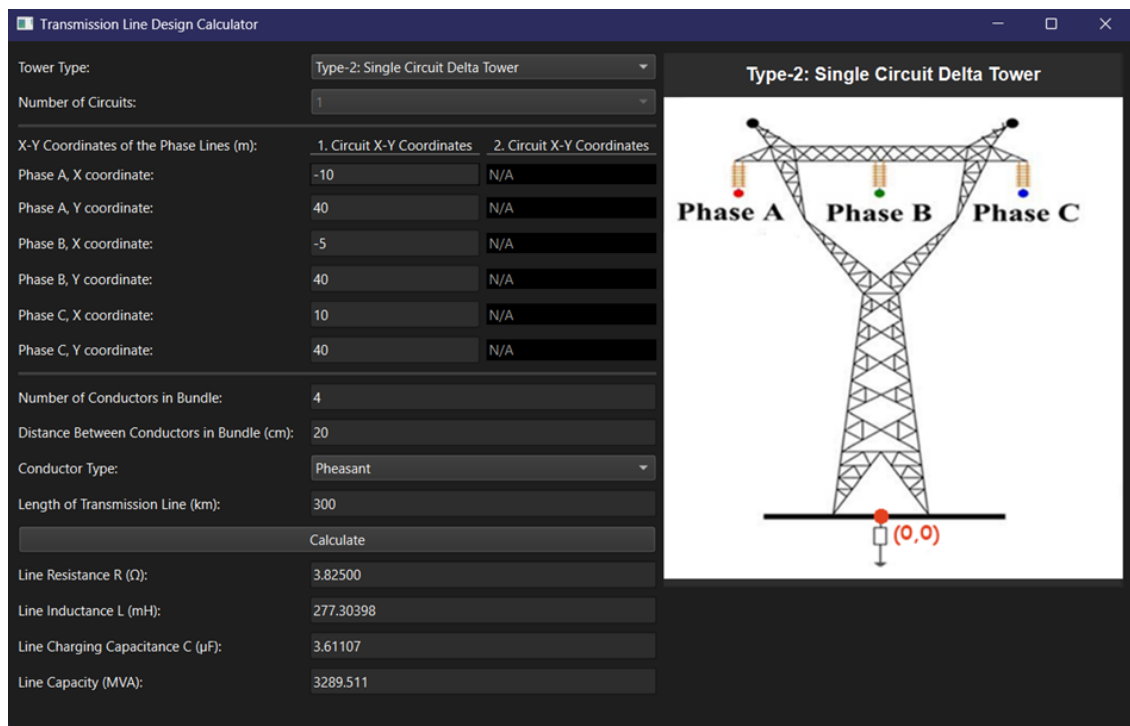


Figure 6. The Results When The Correct Input Is Entered for Type 2- Single Circuit Delta Tower.

Transmission Line Design Calculator

Tower Type: Type-3: Double Circuit Vertical Tower

Number of Circuits: 2

X-Y Coordinates of the Phase Lines (m):

	1. Circuit X-Y Coordinates	2. Circuit X-Y Coordinates
Phase A, X coordinate:	3	-3
Phase A, Y coordinate:	45	45
Phase B, X coordinate:	2	-2
Phase B, Y coordinate:	40	40
Phase C, X coordinate:	4	-4
Phase C, Y coordinate:	36	36

Number of Conductors in Bundle: 1

Distance Between Conductors in Bundle (cm): 20

Conductor Type: Rail

Length of Transmission Line (km): 30

Calculate

Line Resistance R (Ω): 2.04000

Line Inductance L (mH): 19.76606

Line Charging Capacitance C (μ F): 0.50661

Line Capacity (MVA): 529.737

Type-3: Double Circuit Vertical Tower

Figure 7. The Results When the Correct Input Is Entered for Type 3- Double Circuit Vertical Tower.

Transmission Line Design Calculator

Tower Type: Type-3: Double Circuit Vertical Tower

Number of Circuits: 2

X-Y Coordinates of the Phase Lines (m):

	1. Circuit X-Y Coordinates	2. Circuit X-Y Coordinates
Phase A, X coordinate: *	1.8 - 5.35	-5.35 - -1.8
Phase A, Y coordinate: *	36 - 48.8	36 - 48.8
Phase B, X coordinate: *	1.8 - 5.35	-5.35 - -1.8
Phase B, Y coordinate: *	36 - 48.8	36 - 48.8
Phase C, X coordinate: *	1.8 - 5.35	-5.35 - -1.8
Phase C, Y coordinate: *	36 - 48.8	36 - 48.8

Number of Conductors in Bundle: *

Distance Between Conductors in Bundle (cm): *

Conductor Type: Pheasant

Length of Transmission Line (km): *

Calculate

Line Resistance R (Ω): N/A

Line Inductance L (mH): N/A

Line Charging Capacitance C (μ F): N/A

Line Capacity (MVA): N/A

Type-3: Double Circuit Vertical Tower

Invalid Input

Distance Between Conductors in Bundle (cm): must be a number.
Length of Transmission Line (km): must be a positive number.
Invalid number of conductors in bundle.
Invalid coordinates for Phase A.
Invalid coordinates for Phase B.
Invalid coordinates for Phase C.
Invalid coordinates for Phase A in the second circuit.
Invalid coordinates for Phase B in the second circuit.
Invalid coordinates for Phase C in the second circuit.

OK

Figure 8. The Interface and Errors When Negative Number And String Inputs Is Entered.

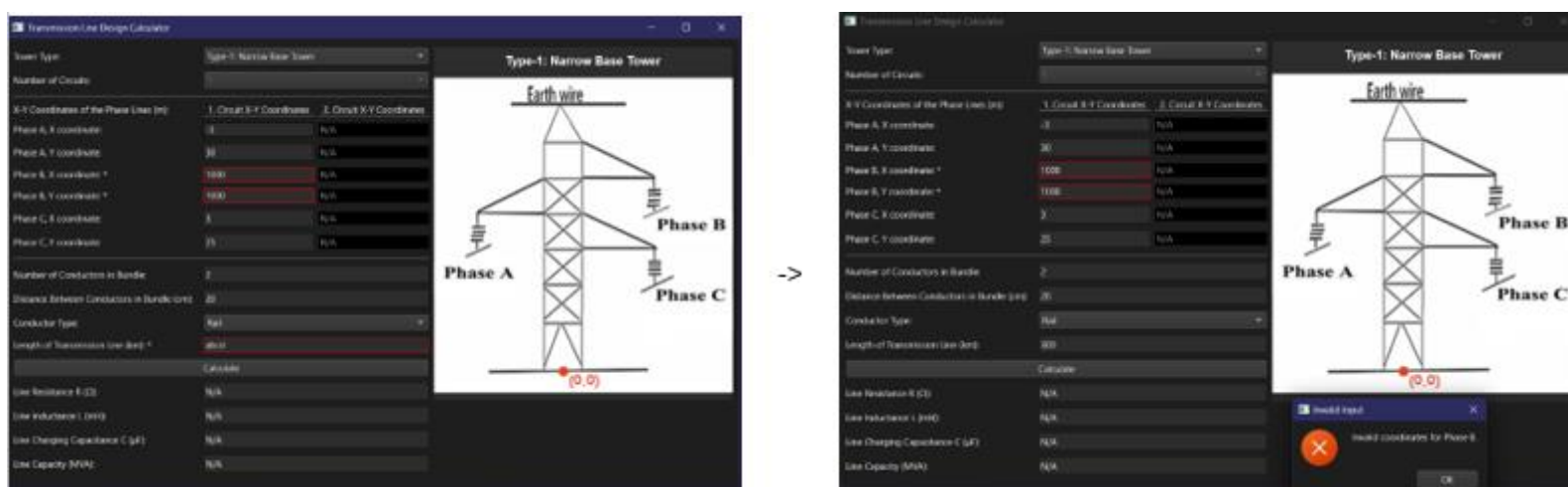


Figure 9. The Updating GUI Interfaces Each Time The Input Changes And Calculate Button Is Clicked.

As test results, as shown in nine figures, the GUI's default interfaces, error messages when incorrect or partially correct inputs are entered, calculations when correct inputs are entered can be seen. In conclusion, this "Transmission Line Tower Design Calculator" GUI is a powerful tool developed to help in precise calculation of line parameters for transmission line and tower designs.

6. Conclusion

The "Transmission Line Tower Design Calculator" is an essential tool in electrical power transmission design. This tool, developed with Python and PyQt, simplifies complex calculations for transmission lines and towers with a user-friendly interface.

It supports three types of transmission towers: Narrow Base Tower (Type-1), Single Circuit Delta Tower (Type-2), and Double Circuit Vertical Tower (Type-3). Each type has specific geometric constraints and voltage levels, with options for single and double circuit configurations, providing different solutions for various transmission needs.

The calculator accurately computes key parameters such as line resistance (R), line inductance (L), line charging capacitance (C), and line capacity (MVA), considering conductor properties like diameter, GMR, AC resistance, and current capacity. Input validation and error handling ensure the reliability of the design process.

The use of "PyInstaller" to create a standalone executable allows for easy distribution and use without Python dependencies for users. Extensive testing has validated the tool's functionality, demonstrating its capability to handle various input scenarios and provide reliable outputs. Overall, the "Transmission Line Tower Design Calculator" streamlines the design process, ensuring accurate results and supporting efficient power transmission project planning.

7. References

1. A. R. Bergen and V. Vittal, Power System Analysis, Prentice Hall, 2nd Edition, 2000.
2. J. J. Grainger and W. D. Stevenson, Power System Analysis, McGraw-Hill, 2016.
3. <https://www.youtube.com/watch?v=WFakOD-DnwU&list=WL&index=7&t=2040s>

8.Appendix

Transmission_line_design_calculator.py

```
1 from PySide6.QtWidgets import *
2 from PySide6.QtGui import *
3 from PySide6.QtCore import *
4 import math
5
6 def set_dark_theme(app):
7     app.setStyle("Fusion")
8
9     dark_palette = QPalette()
10
11     # Base colors
12     dark_palette.setColor(QPalette.Window, QColor(30,30,30))
13     dark_palette.setColor(QPalette.WindowText, QColor(255, 255, 255))
14     dark_palette.setColor(QPalette.Base, QColor(45,45,45))
15     dark_palette.setColor(QPalette.AlternateBase, QColor(60,60,60))
16     dark_palette.setColor(QPalette.ToolTipBase, QColor(255, 255, 255))
17     dark_palette.setColor(QPalette.ToolTipText, QColor(255, 255, 255))
18     dark_palette.setColor(QPalette.PlaceholderText, QColor(156, 155, 152))
19     dark_palette.setColor(QPalette.Text, QColor(255, 255, 255))
20     dark_palette.setColor(QPalette.Button, QColor(45,45,45))
21     dark_palette.setColor(QPalette.ButtonText, QColor(255, 255, 255))
22     dark_palette.setColor(QPalette.BrightText, QColor(255, 0, 0))
23
24     # Highlight colors
25     dark_palette.setColor(QPalette.Highlight, QColor(57,57,57))
26     dark_palette.setColor(QPalette.HighlightedText, QColor(255, 255, 255))
27
28     # Disabled colors
29     dark_palette.setColor(QPalette.Disabled, QPalette.WindowText, QColor(127, 127, 127))
30     dark_palette.setColor(QPalette.Disabled, QPalette.Text, QColor(127, 127, 127))
31     dark_palette.setColor(QPalette.Disabled, QPalette.ButtonText, QColor(127, 127, 127))
32
33     app.setPalette(dark_palette)
34
35 limits = {
36     "Type-1: Narrow Base Tower": {
37         "x_coordinates_a": (-4.0, -2.2),
38         "y_coordinates_a": (23, 39),
39         "x_coordinates_b": (2.2, 4.0),
40         "y_coordinates_b": (23, 39),
41         "x_coordinates_c": (2.2, 4.0),
42         "y_coordinates_c": (23, 39),
43         "number_of_conductors": (1, 3)
44     },
45     "Type-2: Single Circuit Delta Tower": {
46         "x_coordinates_a": (-11.5, -9.4),
47         "y_coordinates_a": (38.25, 43),
48         "x_coordinates_b": (-8.9, 8.9),
49         "y_coordinates_b": (38.25, 43),
50         "x_coordinates_c": (9.4, 11.5),
51         "y_coordinates_c": (38.25, 43),
52         "number_of_conductors": (1,4)
53     },
54     "Type-3: Double Circuit Vertical Tower": {
55
56         "x_coordinates_a": (1.8, 5.35),
57         "y_coordinates_a": (36, 48.8),
```

```

58         "x_coordinates_b": (1.8, 5.35),
59         "y_coordinates_b": (36, 48.8),
60         "x_coordinates_c": (1.8, 5.35),
61         "y_coordinates_c": (36, 48.8),
62         "number_of_conductors": (1,3)
63     }
64 }
65
66 class TransmissionLineGUI(QMainWindow):
67     def __init__(self):
68         super().__init__()
69         self.setWindowTitle("Transmission Line Design Calculator")
70         self.setGeometry(100, 100, 800, 600)
71         self.initUI()
72
73     def initUI(self):
74         widget = QWidget()
75         self.setCentralWidget(widget)
76
77         grid_layout = QGridLayout()
78         widget.setLayout(grid_layout)
79
80         # layout for the input/output fields
81         grid_layout.addWidget(QLabel("Tower Type:"), 0, 0)
82         self.tower_type = QComboBox()
83         self.tower_type.addItem(
84             ["Type-1: Narrow Base Tower", "Type-2: Single Circuit Delta Tower",
85              "Type-3: Double Circuit Vertical Tower"])
86
87         grid_layout.addWidget(self.tower_type, 0, 1, 1, 2)
88
89         grid_layout.addWidget(QLabel("Number of Circuits:"), 1, 0)
90         self.number_of_circuits = QComboBox()
91         self.number_of_circuits.addItem(["1", "2"]) # Added combobox for circuit selection
92         self.number_of_circuits.setEnabled(False) # Disable by default
93         self.number_of_circuits.setToolTip("For Type-1 and Type-3 towers, only single
circuit can be deployed.")
94         grid_layout.addWidget(self.number_of_circuits, 1, 1, 1, 2)
95
96         line_top = QFrame()
97         line_top.setFrameShape(QFrame.HLine)
98         line_top.setFrameShadow(QFrame.Sunken)
99         line_top.setStyleSheet("background-color: rgb(60,60,60);")
100        grid_layout.addWidget(line_top, 2, 0, 1, 3)
101
102        label_xy_coordinates = QLabel("X-Y Coordinates of the Phase Lines (m):")
103        grid_layout.addWidget(label_xy_coordinates, 3, 0)
104        label_xy_coordinates.setSizePolicy(QSizePolicy.MinimumExpanding, QSizePolicy.Fixed)
105        label_xy_coordinates_circ1 = QLabel(" 1. Circuit X-Y Coordinates ")
106        label_xy_coordinates_circ1.setStyleSheet("border: none; border-bottom: 1px groove
gray;")
107        grid_layout.addWidget(label_xy_coordinates_circ1, 3, 1)
108        self.label_xa = QLabel("Phase A, X coordinate:")
109        grid_layout.addWidget(self.label_xa, 4, 0)
110        self.x_coordinates_a = QLineEdit()
111        grid_layout.addWidget(self.x_coordinates_a, 4, 1)
112        self.label_ya = QLabel("Phase A, Y coordinate:")
113        grid_layout.addWidget(self.label_ya, 5, 0)
114        self.y_coordinates_a = QLineEdit()
115        grid_layout.addWidget(self.y_coordinates_a, 5, 1)

```



```

116 self.label_xb = QLabel("Phase B, X coordinate:")
117 grid_layout.addWidget(self.label_xb, 6, 0)
118 self.x_coordinates_b = QLineEdit()
119 grid_layout.addWidget(self.x_coordinates_b, 6, 1)
120 self.label_yb = QLabel("Phase B, Y coordinate:")
121 grid_layout.addWidget(self.label_yb, 7, 0)
122 self.y_coordinates_b = QLineEdit()
123 grid_layout.addWidget(self.y_coordinates_b, 7, 1)
124 self.label_xc = QLabel("Phase C, X coordinate:")
125 grid_layout.addWidget(self.label_xc, 8, 0)
126 self.x_coordinates_c = QLineEdit()
127 grid_layout.addWidget(self.x_coordinates_c, 8, 1)
128 self.label_yc = QLabel("Phase C, Y coordinate:")
129 grid_layout.addWidget(self.label_yc, 9, 0)
130 self.y_coordinates_c = QLineEdit()
131 grid_layout.addWidget(self.y_coordinates_c, 9, 1)
132
133 # Additional fields for second circuit
134 label_xy_coordinates_circ2 = QLabel(" 2. Circuit X-Y Coordinates ")
135 label_xy_coordinates_circ2.setStyleSheet("border: none; border-bottom: 1px groove
gray;")
136 grid_layout.addWidget(label_xy_coordinates_circ2, 3, 2)
137 self.x_coordinates_a_2 = QLineEdit()
138 grid_layout.addWidget(self.x_coordinates_a_2, 4, 2)
139 self.y_coordinates_a_2 = QLineEdit()
140 grid_layout.addWidget(self.y_coordinates_a_2, 5, 2)
141 self.x_coordinates_b_2 = QLineEdit()
142 grid_layout.addWidget(self.x_coordinates_b_2, 6, 2)
143 self.y_coordinates_b_2 = QLineEdit()
144 grid_layout.addWidget(self.y_coordinates_b_2, 7, 2)
145 self.x_coordinates_c_2 = QLineEdit()
146 grid_layout.addWidget(self.x_coordinates_c_2, 8, 2)
147 self.y_coordinates_c_2 = QLineEdit()
148 grid_layout.addWidget(self.y_coordinates_c_2, 9, 2)
149
150 line_bottom = QFrame()
151 line_bottom.setFrameShape(QFrame.HLine)
152 line_bottom.setFrameShadow(QFrame.Sunken)
153 line_bottom.setStyleSheet("background-color: rgb(60,60,60);")
154 grid_layout.addWidget(line_bottom, 10, 0, 1, 3)
155
156 self.label_number_of_conductors = QLabel("Number of Conductors in Bundle:")
157 grid_layout.addWidget(self.label_number_of_conductors, 11, 0)
158 self.number_of_conductors = QLineEdit()
159 grid_layout.addWidget(self.number_of_conductors, 11, 1, 1, 2)
160
161 self.label_distance_between_conductors = QLabel("Distance Between Conductors in
Bundle (cm):")
162 grid_layout.addWidget(self.label_distance_between_conductors, 12, 0)
163 self.distance_between_conductors = QLineEdit()
164 grid_layout.addWidget(self.distance_between_conductors, 12, 1, 1, 2)
165
166 grid_layout.addWidget(QLabel("Conductor Type:"), 13, 0)
167 self.conductor_type = QComboBox()
168 self.conductor_type.addItems(["Hawk", "Drake", "Cardinal", "Rail", "Pheasant"])
169 grid_layout.addWidget(self.conductor_type, 13, 1, 1, 2)
170
171 self.label_line_length = QLabel("Length of Transmission Line (km):")
172 grid_layout.addWidget(self.label_line_length, 14, 0)
173 self.line_length = QLineEdit()

```

```

174         grid_layout.addWidget(self.line_length, 14, 1, 1, 2)
175
176         self.calculate_button = QPushButton("Calculate")
177         grid_layout.addWidget(self.calculate_button, 15, 0, 1, 3)
178         self.calculate_button.clicked.connect(self.calculate_parameters) # calculate the
parameters when the button is clicked
179
180         self.output_R = QLineEdit()
181         self.output_R.setDisabled(True)
182         self.output_R.setStyleSheet("color: white;")
183         grid_layout.addWidget(QLabel("Line Resistance R ( $\Omega$ ):"), 16, 0)
184         grid_layout.addWidget(self.output_R, 16, 1, 1, 2)
185
186         self.output_L = QLineEdit()
187         self.output_L.setDisabled(True)
188         self.output_L.setStyleSheet("color: white;")
189         grid_layout.addWidget(QLabel("Line Inductance L (mH):"), 17, 0)
190         grid_layout.addWidget(self.output_L, 17, 1, 1, 2)
191
192         self.output_C = QLineEdit()
193         self.output_C.setDisabled(True)
194         self.output_C.setStyleSheet("color: white;")
195         grid_layout.addWidget(QLabel("Line Charging Capacitance C ( $\mu$ F):"), 18, 0)
196         grid_layout.addWidget(self.output_C, 18, 1, 1, 2)
197
198         self.output_capacity = QLineEdit()
199         self.output_capacity.setDisabled(True)
200         self.output_capacity.setStyleSheet("color: white;")
201         grid_layout.addWidget(QLabel("Line Capacity (MVA):"), 19, 0)
202         grid_layout.addWidget(self.output_capacity, 19, 1, 1, 2)
203
204         self.image_frame = QFrame()
205         self.image_frame.setFrameShape(QFrame.StyledPanel)
206         self.image_frame.setFrameShadow(QFrame.Raised)
207         self.image_frame.setStyleSheet("background-color: rgb(45,45,45);")
208         self.image_frame.setLineWidth(2)
209         self.image_layout = QVBoxLayout(self.image_frame)
210         grid_layout.addWidget(self.image_frame, 0, 3, 17, 1)
211
212         self.title_label = QLabel("Tower Type") # change the title according to (temporary
name)
213         self.title_label.setAlignment(Qt.AlignTop | Qt.AlignHCenter)
214         self.title_label.setFont(QFont("Arial", 12, QFont.Bold))
215         self.image_layout.addWidget(self.title_label)
216
217         self.image_label = QLabel()
218         grid_layout.addWidget(self.image_label, 0, 3, 18, 1)
219
220         grid_layout.setColumnStretch(0, 1)
221         total_rows = 20
222         for i in range(total_rows):
223             grid_layout.setRowStretch(i, 1)
224
225         self.tower_type.currentIndexChanged.connect(self.update_circuits_input)
226         self.number_of_circuits.currentIndexChanged.connect(self.update_circuits_input)
227         self.update_circuits_input()
228
229     def update_circuits_input(self):
230         # Clear input/output fields
231         coordinates = [

```



```

232         self.x_coordinates_a, self.y_coordinates_a, self.x_coordinates_b,
self.y_coordinates_b,
233         self.x_coordinates_c, self.y_coordinates_c, self.x_coordinates_a_2,
self.y_coordinates_a_2,
234         self.x_coordinates_b_2, self.y_coordinates_b_2, self.x_coordinates_c_2,
self.y_coordinates_c_2,
235         self.number_of_conductors, self.output_R, self.output_L, self.output_C,
self.output_capacity
236     ]
237
238     for coordinate in coordinates:
239         coordinate.clear()
240
241     # Reset styles
242     for coordinate in coordinates:
243         coordinate.setStyleSheet("")
244
245     # Reset labels' styles
246     labels = [
247         self.label_number_of_conductors, self.label_xa, self.label_ya,
248         self.label_xb, self.label_yb, self.label_xc, self.label_yc,
249         self.label_distance_between_conductors, self.label_line_length
250     ]
251
252     for label in labels:
253         label.setText(f"{label.text().strip('* ')}")
254
255     # Reset additional styles
256     self.line_length.setStyleSheet("")
257     self.distance_between_conductors.setStyleSheet("")
258
259
260     # change the limits of the coordinates based on the tower type
261     tower_type = self.tower_type.currentText()
262     limits_dict = limits[tower_type]
263     self.x_coordinates_a.setValidator(QDoubleValidator(limits_dict["x_coordinates_a"][0]
, limits_dict["x_coordinates_a"][1], 2))
264     self.y_coordinates_a.setValidator(QDoubleValidator(limits_dict["y_coordinates_a"][0]
, limits_dict["y_coordinates_a"][1], 2))
265     self.x_coordinates_b.setValidator(QDoubleValidator(limits_dict["x_coordinates_b"][0]
, limits_dict["x_coordinates_b"][1], 2))
266     self.y_coordinates_b.setValidator(QDoubleValidator(limits_dict["y_coordinates_b"][0]
, limits_dict["y_coordinates_b"][1], 2))
267     self.x_coordinates_c.setValidator(QDoubleValidator(limits_dict["x_coordinates_c"][0]
, limits_dict["x_coordinates_c"][1], 2))
268     self.y_coordinates_c.setValidator(QDoubleValidator(limits_dict["y_coordinates_c"][0]
, limits_dict["y_coordinates_c"][1], 2))
269     self.x_coordinates_a_2.setValidator(QDoubleValidator(-1*limits_dict["
x_coordinates_a"][1], -1*limits_dict["x_coordinates_a"][0], 2))
270     self.y_coordinates_a_2.setValidator(QDoubleValidator(limits_dict["y_coordinates_a"]
[0], limits_dict["y_coordinates_a"][1], 2))
271     self.x_coordinates_b_2.setValidator(QDoubleValidator(-1*limits_dict["
x_coordinates_b"][1], -1*limits_dict["x_coordinates_b"][0], 2))
272     self.y_coordinates_b_2.setValidator(QDoubleValidator(limits_dict["y_coordinates_b"]
[0], limits_dict["y_coordinates_b"][1], 2))
273     self.x_coordinates_c_2.setValidator(QDoubleValidator(-1*limits_dict["
x_coordinates_c"][1], -1*limits_dict["x_coordinates_c"][0], 2))
274     self.y_coordinates_c_2.setValidator(QDoubleValidator(limits_dict["y_coordinates_c"]
[0], limits_dict["y_coordinates_c"][1], 2))
275     self.number_of_conductors.setValidator(QIntValidator(limits_dict["
number_of_conductors"][0], limits_dict["number_of_conductors"][1]))
276
277     # change the placeholder text based on the tower type

```

```

278         self.x_coordinates_a.setPlaceholderText(f"{limits_dict['x_coordinates_a'][0]} -
{limits_dict['x_coordinates_a'][1]}")
279         self.y_coordinates_a.setPlaceholderText(f"{limits_dict['y_coordinates_a'][0]} -
{limits_dict['y_coordinates_a'][1]}")
280         self.x_coordinates_b.setPlaceholderText(f"{limits_dict['x_coordinates_b'][0]} -
{limits_dict['x_coordinates_b'][1]}")
281         self.y_coordinates_b.setPlaceholderText(f"{limits_dict['y_coordinates_b'][0]} -
{limits_dict['y_coordinates_b'][1]}")
282         self.x_coordinates_c.setPlaceholderText(f"{limits_dict['x_coordinates_c'][0]} -
{limits_dict['x_coordinates_c'][1]}")
283         self.y_coordinates_c.setPlaceholderText(f"{limits_dict['y_coordinates_c'][0]} -
{limits_dict['y_coordinates_c'][1]}")
284         self.number_of_conductors.setPlaceholderText(f"{limits_dict['number_of_conductors']
[0]} - {limits_dict['number_of_conductors'][1]}")
285
286         # Enable the number of circuits input if the tower type is Type-3
287         if self.tower_type.currentText() == "Type-3: Double Circuit Vertical Tower":
288             self.number_of_circuits.setEnabled(True)
289         else:
290             self.number_of_circuits.setEnabled(False)
291             self.number_of_circuits.setCurrentIndex(0) # Default to 1 circuit when disabled
292
293         #Determine if the second circuit inputs should be changable or not when the number
of circuits is changed
294         if self.number_of_circuits.currentText() == "2":
295             self.x_coordinates_a_2.setReadOnly(False)
296             self.y_coordinates_a_2.setReadOnly(False)
297             self.x_coordinates_b_2.setReadOnly(False)
298             self.y_coordinates_b_2.setReadOnly(False)
299             self.x_coordinates_c_2.setReadOnly(False)
300             self.y_coordinates_c_2.setReadOnly(False)
301             self.x_coordinates_a_2.setStyleSheet("")
302             self.y_coordinates_a_2.setStyleSheet("")
303             self.x_coordinates_b_2.setStyleSheet("")
304             self.y_coordinates_b_2.setStyleSheet("")
305             self.x_coordinates_c_2.setStyleSheet("")
306             self.y_coordinates_c_2.setStyleSheet("")
307             self.x_coordinates_a_2.setPlaceholderText(f"{-1*limits_dict['x_coordinates_a']
[1]} - {-1*limits_dict['x_coordinates_a'][0]}")
308             self.y_coordinates_a_2.setPlaceholderText(f"{limits_dict['y_coordinates_a'][0]}
- {limits_dict['y_coordinates_a'][1]}")
309             self.x_coordinates_b_2.setPlaceholderText(f"{-1*limits_dict['x_coordinates_b']
[1]} - {-1*limits_dict['x_coordinates_b'][0]}")
310             self.y_coordinates_b_2.setPlaceholderText(f"{limits_dict['y_coordinates_b'][0]}
- {limits_dict['y_coordinates_b'][1]}")
311             self.x_coordinates_c_2.setPlaceholderText(f"{-1*limits_dict['x_coordinates_c']
[1]} - {-1*limits_dict['x_coordinates_c'][0]}")
312             self.y_coordinates_c_2.setPlaceholderText(f"{limits_dict['y_coordinates_c'][0]}
- {limits_dict['y_coordinates_c'][1]}")
313         else:
314             self.x_coordinates_a_2.setReadOnly(True)
315             self.y_coordinates_a_2.setReadOnly(True)
316             self.x_coordinates_b_2.setReadOnly(True)
317             self.y_coordinates_b_2.setReadOnly(True)
318             self.x_coordinates_c_2.setReadOnly(True)
319             self.y_coordinates_c_2.setReadOnly(True)
320             self.x_coordinates_a_2.setStyleSheet("background-color: black; border: none;
QlineEdit::placeholder {color: light gray;}")
321             self.y_coordinates_a_2.setStyleSheet("background-color: black; border: none;")
322             self.x_coordinates_b_2.setStyleSheet("background-color: black; border: none;")
323             self.y_coordinates_b_2.setStyleSheet("background-color: black; border: none;")
324             self.x_coordinates_c_2.setStyleSheet("background-color: black; border: none;")
325             self.y_coordinates_c_2.setStyleSheet("background-color: black; border: none;")

```

```

326         self.x_coordinates_a_2.setPlaceholderText("N/A")
327         self.y_coordinates_a_2.setPlaceholderText("N/A")
328         self.x_coordinates_b_2.setPlaceholderText("N/A")
329         self.y_coordinates_b_2.setPlaceholderText("N/A")
330         self.x_coordinates_c_2.setPlaceholderText("N/A")
331         self.y_coordinates_c_2.setPlaceholderText("N/A")
332
333     # Determine the image path based on the tower type
334     if tower_type == "Type-1: Narrow Base Tower":
335         self.image_path = "type1.png"
336         self.title_text = "Type-1: Narrow Base Tower"
337     elif tower_type == "Type-2: Single Circuit Delta Tower":
338         self.image_path = "type2.png"
339         self.title_text = "Type-2: Single Circuit Delta Tower"
340     elif tower_type == "Type-3: Double Circuit Vertical Tower":
341         self.image_path = "type3.png"
342         self.title_text = "Type-3: Double Circuit Vertical Tower"
343     else:
344         self.image_path = "" # Default
345         self.title_text = "Unknown Tower Type"
346     # Set the title label text
347     self.title_label.setText(self.title_text)
348     # Set the image
349     pixmap = QPixmap(self.image_path)
350     self.image_label.setPixmap(pixmap)
351
352     def validate_input(self):
353         inputs = [
354             (self.label_distance_between_conductors, self.distance_between_conductors),
355             (self.label_line_length, self.line_length),
356         ]
357
358         error_messages = []
359
360         # Reset the style of all line edits and in the dictionary
361         for label, line_edit in inputs:
362             line_edit.setStyleSheet("")
363             label.setText(f"{label.text().strip('* ')}")
364         self.label_number_of_conductors.setText(f"{self.label_number_of_conductors.text()
365 .strip('* ')}")
366         self.number_of_conductors.setStyleSheet("")
367         self.label_xa.setText(f"{self.label_xa.text().strip('* ')}")
368         self.x_coordinates_a.setStyleSheet("QLineEdit {color: black;} QLineEdit::placeholder
369 {color: light gray;} QLineEdit {border: 1px solid black;}")
370         self.label_ya.setText(f"{self.label_ya.text().strip('* ')}")
371         self.y_coordinates_a.setStyleSheet("")
372         self.label_xb.setText(f"{self.label_xb.text().strip('* ')}")
373         self.x_coordinates_b.setStyleSheet("")
374         self.label_yb.setText(f"{self.label_yb.text().strip('* ')}")
375         self.y_coordinates_b.setStyleSheet("")
376         self.label_xc.setText(f"{self.label_xc.text().strip('* ')}")
377         self.x_coordinates_c.setStyleSheet("")
378         self.label_yc.setText(f"{self.label_yc.text().strip('* ')}")
379         self.y_coordinates_c.setStyleSheet("")
380         if self.number_of_circuits.currentText() == "2":
381             self.x_coordinates_a_2.setStyleSheet("")
382             self.y_coordinates_a_2.setStyleSheet("")
383             self.x_coordinates_b_2.setStyleSheet("")
384             self.y_coordinates_b_2.setStyleSheet("")
385             self.x_coordinates_c_2.setStyleSheet("")

```

```

384         self.y_coordinates_c_2.setStyleSheet("")
385     else:
386         self.x_coordinates_a_2.setStyleSheet("background-color: black; border: none;")
387         self.y_coordinates_a_2.setStyleSheet("background-color: black; border: none;")
388         self.x_coordinates_b_2.setStyleSheet("background-color: black; border: none;")
389         self.y_coordinates_b_2.setStyleSheet("background-color: black; border: none;")
390         self.x_coordinates_c_2.setStyleSheet("background-color: black; border: none;")
391         self.y_coordinates_c_2.setStyleSheet("background-color: black; border: none;")
392
393     # Check if all inputs are within limits and convert to float (except for
coordinates)
394     for label, line_edit in inputs:
395         value = line_edit.text()
396         if not value:
397             error_messages.append(f"Please enter {label.text()}.")
398             line_edit.setStyleSheet("QlineEdit {color: red;} QlineEdit::placeholder
{color: red;} QlineEdit {border: 1px solid red;}")
399             label.setText(f"{label.text().strip('* ')} *")
400         else:
401             try:
402                 float_value = float(value)
403                 if float_value <= 0:
404                     error_messages.append(f"{label.text()} must be a positive number.")
405                     line_edit.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
406                     label.setText(f"{label.text().strip('* ')} *")
407                 except ValueError:
408                     error_messages.append(f"{label.text()} must be a number.")
409                     line_edit.setStyleSheet("QlineEdit {color: red;} QlineEdit::placeholder
{color: red;} QlineEdit {border: 1px solid red;}")
410                     label.setText(f"{label.text().strip('* ')} *")
411
412     # Checks if the number of conductors in bundle is in the limit and its not empty
413     if not self.number_of_conductors.hasAcceptableInput():
414         error_messages.append("Invalid number of conductors in bundle.")
415         self.label_number_of_conductors.setText(f"
{self.label_number_of_conductors.text().strip('* ')} *")
416         self.number_of_conductors.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
417
418     # check the validity of the coordinates
419     if not (self.x_coordinates_a.hasAcceptableInput() and
self.y_coordinates_a.hasAcceptableInput()):
420         error_messages.append("Invalid coordinates for Phase A.")
421         self.label_xa.setText(f"{self.label_xa.text().strip('* ')} *")
422         self.x_coordinates_a.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
423         self.label_ya.setText(f"{self.label_ya.text().strip('* ')} *")
424         self.y_coordinates_a.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
425     if not (self.x_coordinates_b.hasAcceptableInput() and
self.y_coordinates_b.hasAcceptableInput()):
426         error_messages.append("Invalid coordinates for Phase B.")
427         self.label_xb.setText(f"{self.label_xb.text().strip('* ')} *")
428         self.x_coordinates_b.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
429         self.label_yb.setText(f"{self.label_yb.text().strip('* ')} *")
430         self.y_coordinates_b.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
431     if not (self.x_coordinates_c.hasAcceptableInput() and
self.y_coordinates_c.hasAcceptableInput()):
432         error_messages.append("Invalid coordinates for Phase C.")
433         self.label_xc.setText(f"{self.label_xc.text().strip('* ')} *")

```

```

434         self.x_coordinates_c.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
435         self.label_yc.setText(f"{self.label_yc.text().strip('* ')} *")
436         self.y_coordinates_c.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
437         if self.number_of_circuits.currentText() == "2" and not
(self.x_coordinates_a_2.hasAcceptableInput() and self.y_coordinates_a_2.hasAcceptableInput()
):
438             error_messages.append("Invalid coordinates for Phase A in the second circuit.")
439             self.label_xa.setText(f"{self.label_xa.text().strip('* ')} *")
440             self.x_coordinates_a_2.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
441             self.label_ya.setText(f"{self.label_ya.text().strip('* ')} *")
442             self.y_coordinates_a_2.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
443             if self.number_of_circuits.currentText() == "2" and not
(self.x_coordinates_b_2.hasAcceptableInput() and self.y_coordinates_b_2.hasAcceptableInput()
):
444                 error_messages.append("Invalid coordinates for Phase B in the second circuit.")
445                 self.label_xb.setText(f"{self.label_xb.text().strip('* ')} *")
446                 self.x_coordinates_b_2.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
447                 self.label_yb.setText(f"{self.label_yb.text().strip('* ')} *")
448                 self.y_coordinates_b_2.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
449                 if self.number_of_circuits.currentText() == "2" and not
(self.x_coordinates_c_2.hasAcceptableInput() and self.y_coordinates_c_2.hasAcceptableInput()
):
450                     error_messages.append("Invalid coordinates for Phase C in the second circuit.")
451                     self.label_xc.setText(f"{self.label_xc.text().strip('* ')} *")
452                     self.x_coordinates_c_2.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
453                     self.label_yc.setText(f"{self.label_yc.text().strip('* ')} *")
454                     self.y_coordinates_c_2.setStyleSheet("QlineEdit {color: red;}
QlineEdit::placeholder {color: red;} QlineEdit {border: 1px solid red;}")
455
456             # Show error messages if any
457             if error_messages:
458                 QMessageBox.critical(self, "Invalid Input", "\n".join(error_messages))
459             return False
460
461         return True
462
463     def get_conductor_parameters(self): # Returns the conductor parameters based on the
selected type
464         if self.conductor_type.currentText() == "Hawk":
465             diameter = 21.793 * 10**(-3) # in m
466             radius = diameter / 2 # in m
467             r_GMR = 8.809 * 10**(-3) # in m
468             ac_resistance = 0.132 # in ohm/km
469             current_capacity = 659 # in A
470             return radius, r_GMR, ac_resistance, current_capacity
471
472         elif self.conductor_type.currentText() == "Drake":
473             diameter = 28.143 * 10**(-3) # in m
474             radius = diameter / 2 # in m
475             r_GMR = 11.369 * 10**(-3) # in m
476             ac_resistance = 0.080 # in ohm/km
477             current_capacity = 907 # in A
478             return radius, r_GMR, ac_resistance, current_capacity
479
480         elif self.conductor_type.currentText() == "Cardinal":
481             diameter = 30.378 * 10**(-3) # in m

```



```

482         radius = diameter / 2          # in m
483         r_GMR = 12.253 * 10**(-3)      # in m
484         ac_resistance = 0.067          # in ohm/km
485         current_capacity = 996         # in A
486         return radius, r_GMR, ac_resistance, current_capacity
487
488     elif self.conductor_type.currentText() == "Rail":
489         diameter = 29.591 * 10**(-3)    # in m
490         radius = diameter / 2          # in m
491         r_GMR = 11.765 * 10**(-3)      # in m
492         ac_resistance = 0.068          # in ohm/km
493         current_capacity = 993         # in A
494         return radius, r_GMR, ac_resistance, current_capacity
495
496     elif self.conductor_type.currentText() == "Pheasant":
497         diameter = 35.103 * 10**(-3)    # in m
498         radius = diameter / 2          # in m
499         r_GMR = 14.204 * 10**(-3)      # in m
500         ac_resistance = 0.051          # in ohm/km
501         current_capacity = 1187        # in A
502         return radius, r_GMR, ac_resistance, current_capacity
503
504     def calculate_parameters(self):
505         # Check if the inputs are valid
506         if not self.validate_input():
507             self.output_R.setText("N/A")
508             self.output_L.setText("N/A")
509             self.output_C.setText("N/A")
510             self.output_capacity.setText("N/A")
511             return
512
513         # Set the input values for calculation
514         conductor_radius, conductor_GMR, conductor_resistance, conductor_capacity =
self.get_conductor_parameters()
515         distance_between_conductors = float(self.distance_between_conductors.text()) / 100
# Convert cm to m
516         line_length_km = float(self.line_length.text())
517         number_of_conductors = int(self.number_of_conductors.text())
518         x_coordinates_a = float(self.x_coordinates_a.text())
519         y_coordinates_a = float(self.y_coordinates_a.text())
520         x_coordinates_b = float(self.x_coordinates_b.text())
521         y_coordinates_b = float(self.y_coordinates_b.text())
522         x_coordinates_c = float(self.x_coordinates_c.text())
523         y_coordinates_c = float(self.y_coordinates_c.text())
524
525         if self.number_of_circuits.currentText() == "2":
526             x_coordinates_a_2 = float(self.x_coordinates_a_2.text())
527             y_coordinates_a_2 = float(self.y_coordinates_a_2.text())
528             x_coordinates_b_2 = float(self.x_coordinates_b_2.text())
529             y_coordinates_b_2 = float(self.y_coordinates_b_2.text())
530             x_coordinates_c_2 = float(self.x_coordinates_c_2.text())
531             y_coordinates_c_2 = float(self.y_coordinates_c_2.text())
532
533         # Compute Bundle Geometric Mean Radius (GMR)
534         if number_of_conductors == 4:
535             bundle_GMR = (conductor_GMR * (distance_between_conductors ** 3) * math.sqrt(2))
** (1/4) # in m
536             r_eq_bundle = (conductor_radius * (distance_between_conductors ** 3) *
math.sqrt(2)) ** (1/4) # in m
537         else:

```

```

538     bundle_GMR = (conductor_GMR * (distance_between_conductors **
(number_of_conductors - 1))) ** (1/number_of_conductors) # in m
539     r_eq_bundle = (conductor_radius * (distance_between_conductors **
(number_of_conductors - 1))) ** (1/number_of_conductors) # in m
540
541     # Compute distances between phases and GMD values
542     if self.tower_type.currentText() == "Type-1: Narrow Base Tower" or
self.tower_type.currentText() == "Type-2: Single Circuit Delta Tower":
543         Dab = math.sqrt((y_coordinates_b - y_coordinates_a)**2 + (x_coordinates_b + (-
x_coordinates_a))**2)
544         Dbc = math.sqrt((y_coordinates_c - y_coordinates_b)**2 + (x_coordinates_c -
x_coordinates_b)**2)
545         Dca = math.sqrt((y_coordinates_a - y_coordinates_c)**2 + ((-x_coordinates_a) +
x_coordinates_c)**2)
546         gmd = (Dab * Dbc * Dca) ** (1 / 3)
547     else:
548         if self.number_of_circuits.currentText() == "1":
549             Dab = math.sqrt((y_coordinates_a - y_coordinates_b)**2 + (x_coordinates_b -
x_coordinates_a)**2)
550             Dbc = math.sqrt((y_coordinates_b - y_coordinates_c)**2 + (x_coordinates_c -
x_coordinates_b)**2)
551             Dca = math.sqrt((y_coordinates_a - y_coordinates_c)**2 + (x_coordinates_a -
x_coordinates_c)**2)
552             gmd = (Dab * Dbc * Dca) ** (1 / 3)
553         else: # For double circuit
554             #self gmr of phase a,b,c to find new equivalent gmr
555             Dsa = math.sqrt(bundle_GMR * (math.sqrt((y_coordinates_a -
y_coordinates_a_2)**2 + (x_coordinates_a - x_coordinates_a_2)**2)))
556             Dsb = math.sqrt(bundle_GMR * (math.sqrt((y_coordinates_b -
y_coordinates_b_2)**2 + (x_coordinates_b - x_coordinates_b_2)**2)))
557             Dsc = math.sqrt(bundle_GMR * (math.sqrt((y_coordinates_c -
y_coordinates_c_2)**2 + (x_coordinates_c - x_coordinates_c_2)**2)))
558             bundle_GMR = (Dsa * Dsb * Dsc) ** (1/3)
559             Dsa_r = math.sqrt(r_eq_bundle * (math.sqrt((y_coordinates_a -
y_coordinates_a_2)**2 + (x_coordinates_a - x_coordinates_a_2)**2)))
560             Dsb_r = math.sqrt(r_eq_bundle * (math.sqrt((y_coordinates_b -
y_coordinates_b_2)**2 + (x_coordinates_b - x_coordinates_b_2)**2)))
561             Dsc_r = math.sqrt(r_eq_bundle * (math.sqrt((y_coordinates_c -
y_coordinates_c_2)**2 + (x_coordinates_c - x_coordinates_c_2)**2)))
562             r_eq_bundle = (Dsa_r * Dsb_r * Dsc_r) ** (1/3)
563
564             #mutual GMDs of ab, bc, ca to find GMD      (ab distance * ab' distance)^1/2
etc
565             Dab = math.sqrt(math.sqrt((y_coordinates_a - y_coordinates_b)**2 +
(x_coordinates_b - x_coordinates_a)**2) * math.sqrt((y_coordinates_a - y_coordinates_b_2)**2
+ (x_coordinates_a - x_coordinates_b_2)**2))
566             Dbc = math.sqrt(math.sqrt((y_coordinates_b - y_coordinates_c)**2 +
(x_coordinates_c - x_coordinates_b)**2) * math.sqrt((y_coordinates_b - y_coordinates_c_2)**2
+ (x_coordinates_b - x_coordinates_c_2)**2))
567             Dca = math.sqrt(math.sqrt((y_coordinates_a - y_coordinates_c)**2 +
(x_coordinates_c - x_coordinates_a)**2) * math.sqrt((y_coordinates_a - y_coordinates_c_2)**2
+ (x_coordinates_a - x_coordinates_c_2)**2))
568             gmd = (Dab * Dbc * Dca) ** (1 / 3)
569
570     # Calculate parameters
571     R = conductor_resistance / number_of_conductors # Resistance in ohms per km
572     L = (2e-7) * math.log(gmd / bundle_GMR) * 1e3 # Inductance in mH per meter
573     C = (2 * math.pi * 8.854e-12) / math.log(gmd / bundle_GMR) * 1e6 # Capacitance in
µF per meter
574
575     # Calculate total parameters for the line length
576     total_R = R * line_length_km
577     total_L = L * line_length_km * 1000 # since L is per meter and line length is in km
578     total_C = C * line_length_km * 1000 # since C is per meter and line length is in km
579

```

```

580         # Calculate the capacity of the line
581         if self.tower_type.currentText() == "Type-1: Narrow Base Tower":
582             output_capacity = math.sqrt(3) * conductor_capacity * number_of_conductors *
66000 / 1000000 # in MVA
583         elif self.tower_type.currentText() == "Type-2: Single Circuit Delta Tower":
584             output_capacity = math.sqrt(3) * conductor_capacity * number_of_conductors *
400000 / 1000000 # in MVA
585         elif self.tower_type.currentText() == "Type-3: Double Circuit Vertical Tower" and
self.number_of_circuits.currentText() == "1":
586             output_capacity = math.sqrt(3) * conductor_capacity * number_of_conductors *
154000 / 1000000 # in MVA
587         else:
588             output_capacity = math.sqrt(3) * conductor_capacity * number_of_conductors * 2 *
154000 / 1000000 # in MVA for double circuit (*2) since there are 2 parallel circuits
589
590         # Display the results
591         self.output_R.setText(f"{total_R:.5f}")
592         self.output_L.setText(f"{total_L:.5f}")
593         self.output_C.setText(f"{total_C:.5f}")
594         self.output_capacity.setText(f"{output_capacity:.3f}")
595
596     if __name__ == "__main__":
597         app = QApplication([])
598
599         #set dark mode
600         set_dark_theme(app)
601
602         window = TransmissionLineGUI()
603         window.show()
604         app.exec()
605

```